

SSTREAM_REFERENCE

Core Definition

<sstream> provides string-backed streams: `istringstream` (parse from a string), `ostringstream` (build a string), and `stringstream` (both), over `stringbuf`. They are the standard tool for in-memory formatting and parsing — the same `<</>>` interface as files and console, but the "device" is a `std::string`.

#CPP #SSTREAM #STRINGSTREAM #PARSING #FORMATTING #ISTRINGSTREAM #OSTRINGSTREAM

CLASS HIERARCHY

<code>basic_istream</code> — <code>basic_istringstream<CharT></code>	<code>istringstream</code> / <code>wistringstream</code>
<code>basic_ostream</code> — <code>basic_ostringstream<CharT></code>	<code>ostringstream</code> / <code>wostringstream</code>
<code>basic_iostream</code> — <code>basic_stringstream<CharT></code>	<code>stringstream</code> / <code>wstringstream</code>
<code>basic_stringbuf</code> — <code>basic_stringbuf<CharT></code>	<code>stringbuf</code> / <code>wstringbuf</code>

COMPLETE SSTREAM QUICK REFERENCE

```
// =====
// CONSTRUCTION
// =====
std::ostringstream os;           // none      | Empty output stream | Build strings
std::ostringstream os(mode);    // openmode  | With mode          | e.g. ios::ate to append
std::ostringstream os(str);     // string    | Seeded with content | Overwrites from pos 0 unless ate
std::ostringstream os(str, mode); // + openmode | Seeded with mode   |
std::istringstream is(str);     // string    | Parse this string   | Reads from the copy
std::istringstream is(str, mode); // + openmode | With mode          |
std::stringstream ss;          // none      | Read AND write     | Bidirectional
std::stringstream ss(str);      // string    | Seeded, read/write |
std::stringstream os(std::move(other)); // rvalue   | Move construct     | C++11
std::stringstream os(sv, mode); // string_view | From a view        | C++26

// =====
// CONTENT ACCESS
// =====
ss.str()                        // none      | Get contents as string | Returns a COPY (allocates)
ss.str(newStr)                  // string    | Replace contents       | Returns void; resets
    positions
ss.str(std::move(newStr))       // rvalue    | Move contents in      | C++20; avoids a copy
std::move(ss).str()             // rvalue this | Move contents OUT     | C++20; no copy, empties the
    stream
ss.view()                       // none      | Non-owning view       | C++20; no allocation
ss.rdbuf()                      // none      | Underlying stringbuf  | Returns basic_stringbuf*

// =====
// STRINGBUF DIRECT
// =====
std::stringbuf sb;              // none      | Raw string buffer     | Usable with any stream
std::stringbuf sb(str, mode);   // string+mode | Seeded buffer         |
sb.str() / sb.str(s) / sb.view() // Same content interface as the streams
sb.pubseekoff(off, dir, which)  // Seek relative | which: ios::in | ios::out
sb.pubseekpos(pos, which)       // Seek absolute
std::ostream os(&sb);           // Attach a stream to a stringbuf

// =====
// EVERYTHING ELSE IS INHERITED
```

```
// =====
ss << value / ss >> value           // Formatted I/O – all manipulators apply
std::getline(ss, line[, delim])     // Line/field extraction
ss.get / peek / ignore / read / write // Unformatted I/O
ss.tellg / seekg / tellp / seekp    // Positioning
ss.good / eof / fail / bad / clear  // State – see IOSTREAM_REFERENCE
```

COMMON PATTERNS & EXAMPLES

Building a String from Mixed Types

```
#include <sstream>
#include <iomanip>
#include <string>

std::string describe(const std::string& name, int id, double score) {
    std::ostringstream os;
    os << name << " (" << id << ") scored "
    << std::fixed << std::setprecision(2) << score;
    return os.str();
}
// C++20 alternative, usually clearer and faster:
// return std::format("{} (#{}) scored {:.2f}", name, id, score);
```

Parsing a Line into Fields

```
#include <sstream>
#include <string>

struct Record { std::string name; int qty; double price; };

bool parse(const std::string& line, Record& r) {
    std::istringstream is(line);
    if (!(is >> r.name >> r.qty >> r.price)) return false;

    // Reject trailing junk
    std::string extra;
    return !(is >> extra);
}
```

Splitting on a Delimiter

```
std::vector<std::string> split(const std::string& s, char delim) {
    std::vector<std::string> out;
    std::istringstream is(s);
    std::string field;
    while (std::getline(is, field, delim)) out.push_back(field);
    return out;
}
```

Tokenizing on Whitespace

```
std::istringstream is(" alpha bravo\tcharlie\n");
std::string tok;
while (is >> tok) { /* alpha, bravo, charlie – all whitespace collapsed */ }
```

Number ↔ String Conversion

```
// String → number
int toInt(const std::string& s) {
    std::istringstream is(s);
    int v;
    if (!(is >> v)) throw std::invalid_argument("not a number: " + s);
    return v;
}

// Number → string
template <typename T>
std::string toStr(const T& v) {
    std::ostringstream os;
    os << v;
    return os.str();
}

// Prefer std::to_string / std::stoi for simple cases, and
// std::to_chars / std::from_chars (<charconv>) for speed and locale independence.
```

Reusing a Stream (the two-step reset)

```
std::ostringstream os;
for (const auto& item : items) {
    os.str(""); // Clear the contents
    os.clear(); // Clear the state flags – BOTH are required
    os << item;
    send(os.str());
}

// A fresh ostringstream per iteration is often clearer and, with SSO
// and move semantics, rarely slower.
```

Reading Two Ways from One Line

```
std::string line = "SET brightness 75";
std::istringstream is(line);

std::string cmd;
is >> cmd; // "SET"

if (cmd == "SET") {
    std::string key; int value;
    is >> key >> value; // Continues from where the first read stopped
}
```

Efficient Content Access (C++20)

```
std::ostringstream os;
os << "large payload ...";

std::string_view sv = os.view(); // No allocation – valid until os is modified
std::string owned = std::move(os).str(); // Moves the buffer out; os is left empty

// Pre-C++20, os.str() always copied the entire buffer.
```

Capturing Stream Output for Tests

```
#include <sstream>
#include <iostream>

std::string captureCout(void (*fn)()) {
    std::ostringstream capture;
    std::streambuf* old = std::cout.rdbuf(capture.rdbuf());
    fn();
    std::cout.rdbuf(old);          // Always restore
    return capture.str();
}
```

Hex / Binary Parsing

```
unsigned value;
std::istringstream is("1A2B");
is >> std::hex >> value;          // 6699

// With a 0x prefix, std::hex still parses it correctly:
std::istringstream is2("0xFF");
unsigned v2; is2 >> std::hex >> v2;  // 255
```

Reading a Whole Stream into a String

```
#include <fstream>
#include <sstream>

std::ifstream in("file.txt");
std::ostringstream ss;
ss << in.rdbuf();                // Buffer-to-buffer, no per-character loop
std::string contents = ss.str();
```

Nested Parsing (lines then fields)

```
std::istringstream lines(text);
std::string line;
while (std::getline(lines, line)) {
    std::istringstream fields(line);
    std::string field;
    while (std::getline(fields, field, ',')) {
        // process field
    }
}
```

stringstream for Bidirectional Use

```
std::stringstream ss;
ss << 42 << ' ' << 3.14;        // Write

int i; double d;
ss >> i >> d;                    // Read back – the get position starts at 0

// Careful: get and put positions are independent. After writing, reading
// starts at the beginning unless you seek.
```

IMPORTANT CONCEPTS

`str()` Copies (Before C++20)

Every call to `str()` allocates and copies the whole buffer. In a loop this dominates the cost. C++20's `view()` (read-only, no copy) and `std::move(ss).str()` (move out) fix this.

Reset Needs Both `str("")` and `clear()`

`str("")` empties the content but leaves `eofbit` / `failbit` set from a previous read. `clear()` resets the flags but leaves the content. Reusing a stream requires both — and constructing a fresh one avoids the question entirely.

Get and Put Positions Are Separate

In a `stringstream`, `tellg()/seekg()` and `tellp()/seekp()` track different cursors into the same buffer. Writing does not move the read cursor, and vice versa.

`str(s)` Resets Positions

Assigning new content sets both cursors back to the start (or to the end, if the stream was opened with `ios::ate`).

`>>` Skips Whitespace, `getline` Does Not

`is >> field` collapses runs of whitespace and never returns an empty token. `std::getline(is, field, ',', '')` preserves empty fields — which is what CSV parsing needs.

Checking for Complete Consumption

A successful `>>` does not mean the whole input was valid. `"12abc"` parses as `12`. To require full consumption, read a trailing token and confirm it fails, or use `is >> std::ws && is.eof()`.

Locale Dependence

Stream parsing and formatting honor the imbued locale, so a decimal comma or digit grouping can change results. `<charconv>`'s `from_chars` / `to_chars` are locale-independent and belong in serialization paths.

Performance vs `std::format` and `<charconv>`

Stringstreams carry locale lookup, virtual dispatch through `streambuf`, and allocation. For simple conversions `std::to_chars` / `from_chars` are typically several times faster, and `std::format` (C++20) is both faster and more readable than a chain of `<<`. Stringstreams remain the right tool when you need the full operator<< ecosystem, including user-defined types.

`stringstream` Is Gone

The old `<stringstream>` header (`istrstream`, `ostrstream`) was deprecated in C++98 and removed in C++26. Use `<sstream>`, or `<spanstream>` (C++23) when you need a fixed external buffer with no allocation.

CHOOSING A TOOL

Task	Best choice
int/double → string, fast, no locale	std::to_chars (<charconv>)
string → int/double, fast, no locale	std::from_chars (<charconv>)
Simple number → string	std::to_string
Simple string → number	std::stoi / std::stod
Formatted text with placeholders	std::format (C++20)
Building text from user-defined types	std::ostringstream
Parsing whitespace/delimiter-separated	std::istringstream
Fixed external buffer, no allocation	std::spanstream (C++23)
Whole-file read	os << in.rdbuf()

BEST PRACTICES

1. Prefer `std::format` / `<charconv>` for new conversion code; use stringstreams for the `operator<<` ecosystem
2. Use `view()` (C++20) instead of `str()` when you only need to read
3. Use `std::move(ss).str()` to take ownership of the buffer without a copy
4. `str("")` and `clear()` together, or construct a new stream
5. Use `getline` with a delimiter for CSV — `>>` silently drops empty fields
6. Verify full consumption when a parse must be strict
7. `reserve` on the result string is not possible directly — build into a `std::string` with `+=` when the format is simple
8. Construct `istringstream` from the string directly — don't build a temporary just to parse it
9. Restore `rdbuf()` whenever you redirect a standard stream
10. Watch locale effects when parsing numbers from external data

RELATED HEADERS

```
#include <sstream>           // istringstream, ostringstream, stringstream, stringbuf
#include <spanstream>        // C++23: streams over a fixed span, no allocation
#include <syncstream>        // C++20: osyncstream for thread-safe output
#include <charconv>          // C++17: to_chars / from_chars — fast, locale-free
#include <format>            // C++20: std::format
#include <string>            // std::string, to_string, stoi
#include <string_view>       // C++17: view() return type
#include <iomanip>           // manipulators used while building
```

EXTERNAL RESOURCES

- `<sstream>`: <https://en.cppreference.com/w/cpp/header/sstream>
- `basic_stringstream`: https://en.cppreference.com/w/cpp/io/basic_stringstream
- `basic_stringbuf`: https://en.cppreference.com/w/cpp/io/basic_stringbuf

Standard: C++11 (move) / C++20 (`view()`, `move-str()`) / C++23 (`spanstream` replacement) **Last Updated:** September 2026